

INSTITUTO TECNOLÓGICO Y DE ESTUDIOS SUPERIORES DE OCCIDENTE

APRENDIZAJE PROFUNDO



PROYECTO 1 – GENERACIÓN DE IMÁGENES: AUTOENCODERS

Presenta:

Luis Eduardo Jiménez del Muro – 746484 – eduardo.jimenez@iteso.mx
Santiago Reyes Castillo – 745826 – santiago.reyes@iteso.mx
Santiago Montes de Oca – 745377 – smontesdeoca.-@iteso.mx

Profesor: Iván Reyes Amezcua
Fecha: 03/03/2026

Contenido

1. Introducción	3
2 Objetivo	3
3. Marco Teórico.....	3
3.1 Auto-Encoder	3
3.2 Auto-Encoder Variacional	5
3.3 β -Autoencoder.....	6
4. Metodología.....	7
4.1 Exploración y preprocesamiento de datos	7
4.2 Desarrollo inicial de Variational Autoencoders.....	7
4.2 Arquitectura del modelo final.....	9
5. Resultados.....	10
5.1 Funciones de pérdida.....	10
5.2 Reconstrucción de imágenes del set de pruebas	11
5.2 Generación de nuevas imágenes	12
5.3 Ausencia de color y desenfoque	13
6. Conclusión.....	13
7. Referencias:	14
8. Recursos:	14

1. Introducción

En la clase de deep learning los estudiantes hemos visto y entrenado modelos básicos de autoencoders, con diferentes variaciones o ejecuciones. Dentro de este proyecto se, busca que propongamos un dataset y entrenar una red neuronal capaz de generar imágenes con base a las herramientas dadas. Armados con el conocimiento básico, la entrega de este proyecto es una exploración de nuestra capacidad como estudiantes para poder aplicar conocimientos más lejos de lo que llegamos a observar en clase.

En este reporte se detalla la exploración que existió para entender, guiar y aprender respecto a cómo diseñar correctamente el modelo mencionad, la implicación de las fórmulas y el resultado que tienen diferentes combinaciones de hiperparámetros. Al igual que demostrar la capacidad para mejorar continuamente para llegar a un mejor modelo. Se abordará como fue el proceso de reconstrucción de imágenes y generación de nuevas imágenes, así como también las limitaciones que presentó el modelo.

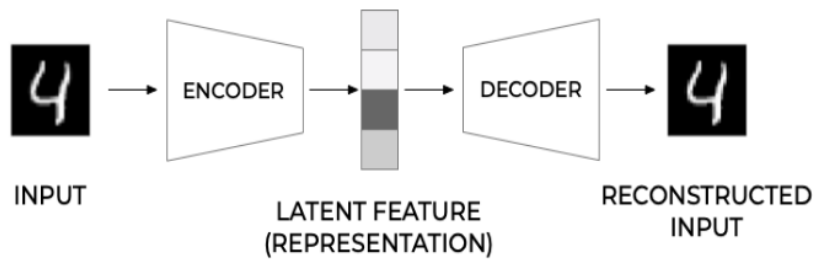
2 Objetivo

El objetivo principal de este proyecto es el de implementar Deep Learning, en la forma de Autoencoders o alguna de sus versiones para ser capaces de obtener un modelo de aprendizaje no supervisado capaz de generar imágenes, a partir de un conjunto de datos de entrenamiento previo. Para esto utilizamos distintas herramientas como los son las librerías de “tensorflow” y “keras” disponibles de manera abierta en Python y un conjunto de imágenes para el entrenamiento del modelo a través de la plataforma de “Kaggle”.

3. Marco Teórico

3.1 Auto-Encoder

Un autoencoder consiste en una red neuronal, que está diseñada para aprender representaciones e información, comprimiéndola en un espacio dimensional mucho más pequeño, llamado espacio latente, para volver a reconstruir dicha entrada de la manera más cercana posible.



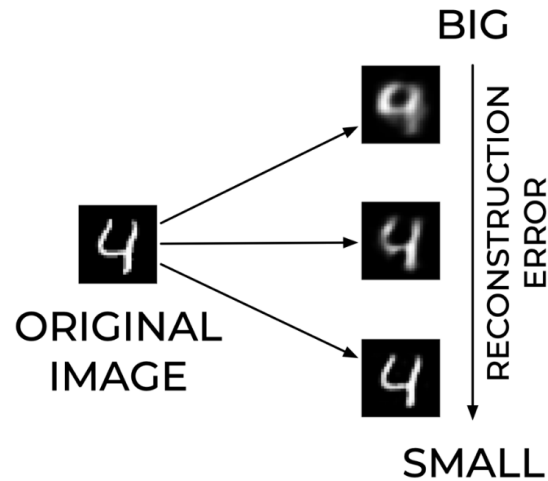
An introduction to autoencoders (Umberto Michelucci).

La arquitectura de un autoencoder es relativamente sencilla, ya que se compone de tres componentes principales. El primero de ellos es el encoder o codificador. Esta sección del modelo transforma la representación de la entrada en una representación comprimida, reduciendo su dimensionalidad, lo que fuerza que el mismo modelo transforme la información para tener solo las características más importantes de la entrada en una dimensionalidad más reducida.

Luego del encoder, tenemos el espacio latente. Una vez que se comprimió la representación de entrada, el espacio latente es un punto medio en el proceso de deconstrucción y construcción de la entrada. En él, se limita arbitrariamente la dimensionalidad para mantener solo la información más relevante para el modelo, y sirve como puente entre el codificador y decodificador.

Al final de la red existe el decoder o decodificador. La finalidad de este componente del modelo es reconstruir la entrada que actualmente se encuentra en el espacio latente, en la que era antes de pasar por el codificador, expandiendo las dimensiones a las que se le redujo en la primera sección del modelo.

El resultado del autoencoder debería ser una entrada-salida muy parecida, en la que el modelo minimizará una función de pérdida como un error cuadrático medio (MSE) o una Entropía-Cruzada Binaria (Binary Cross-Entropy) con el objetivo de medir que tan diferentes son entradas y salidas entre sí, y mientras más parecida sea la entrada a la salida, mejor podemos decir que el modelo está actuando.

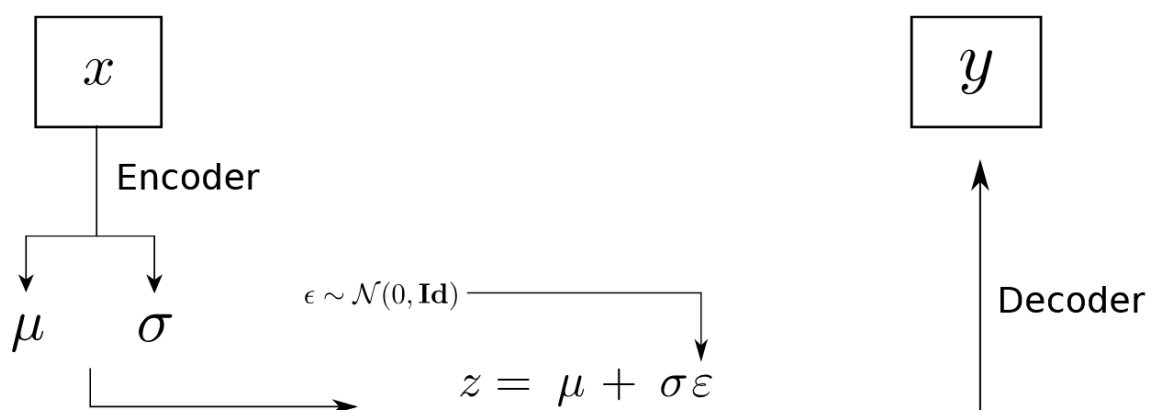


Ejemplo de error de reconstrucción grande contra pequeño en reconstrucción de imágenes

3.2 Auto-Encoder Variacional

Un autoencoder variacional ahonda en la arquitectura de un autoencoder con un objetivo en específico, generación. Un VAE (Variational Auto Encoder) plantea transformar el espacio latente en un espacio probabilístico con una distribución normal. Esto con el objetivo de que, sabiendo que, si las representaciones latentes de las entradas del modelo tienen una distribución normal entre el codificador y decodificador, podemos “samplear” a partir de esta distribución dentro del espacio latente y con esa representación aleatoria que hagamos del espacio latente, el codificador sea capaz de reconstruir una salida nueva, con las características de la información que entrene al modelo.

Con esta arquitectura podemos generar imágenes, texto, sonido, etc., a partir de las estimaciones que hace el encoder de los parámetros para la distribución que mejor se adhiera a los datos una vez estando en el espacio latente.



Variational Autoencoders in Practice (Alasdair Newson)

En el diagrama superior podemos ver que ahora, la responsabilidad del encoder es obtener los parámetros μ y σ que utilizaremos para la distribución de “z”, en el espacio latente. A este “z” le agregamos un componente de error que representa el ruido que tiene la aleatorización de una distribución normal, y luego el decoder genera una salida totalmente nueva llamada “y”.

Sin embargo, el planteamiento de la función de pérdida es distinto, ya que por una parte se busca que el modelo sea capaz de generar salidas parecidas a las entradas, como el autoencoder tradicional, pero al mismo tiempo se busca también que la distribución parametrizada por el encoder se acerque lo más posible a una distribución normal.

$$\mathcal{L}_{\text{VAE}} = \underbrace{\frac{1}{mn} \sum_{i=1}^{mn} (x_i - y_i)^2}_{\text{Reconstruction Loss (MSE)}} + \underbrace{\frac{1}{2} \sum_{j=1}^d (1 + \log(\sigma_j^2) - \mu_j^2 - \sigma_j^2)}_{\text{KL Divergence}}$$

Con el objetivo de que el modelo mejore en ambos aspectos, se introduce un nuevo componente a la pérdida llamada “KL-Divergence” (Kullback Leibler divergence). Este último componente se encarga de medir cuán diferentes son dos distribuciones entre ellas. Entonces, la función de pérdida de nuestro VAE se compone por un elemento que mide el parecido entre entradas y salidas (error de reconstrucción como MSE) y un elemento que mide la similitud entre la distribución parametrizada por el modelo y una distribución normal (KL divergence). Así, el entrenamiento del modelo busca generar representaciones iguales, en entrada y salida, y parametrizar las variables del problema de la manera más cercana a una distribución normal, para luego “samplear” en esta distribución localizada en el espacio latente, y generar representaciones totalmente nuevas a través de la reconstrucción del decoder.

3.3 β -Autoencoder

Uno de los problemas que surge con los VAE es que, cuando reducimos la dimensionalidad de la representación de entrada, para tener un espacio latente más pequeño, muchos de los factores importantes de las entradas se ven entrelazados en los valores de la representación latente. Esto sucede ya que el modelo intenta encapsular todos los factores de la entrada de una manera más reducida, y para mantener más limitada la dimensión de ella, entrelaza la información que mete en el espacio latente. Esto genera un problema al momento de generar nuevos elementos, ya que es imposible controlar factores específicos sin manipular el resto, y de la

misma forma es imposible predecir en qué medida afectarán los cambios hechos dentro de la generación en el espacio latente.

Como solución a este problema, se propuso un marco de mejora al modelo que a través de la implementación de un hiper-parámetro β , limitemos este entrelazamiento de los factores dentro del entrenamiento del modelo. Este factor se implementa como un coeficiente para el componente de la función de pérdida correspondiente a la KL divergence, que se encarga de procurar que la distribución del espacio latente se asemeje a una normal, de la siguiente manera:

$$\text{Total Loss} = \text{Reconstruction Loss} + \beta \times \text{KL Divergence}$$

De esta manera, podemos controlar el esfuerzo de modelo por priorizar una u otra pérdida. En términos prácticos, un valor bajo para el hiperparámetro β el modelo prioriza la reconstrucción, y no procura evitar entrelazar las dimensiones dentro del modelo. Un valor bajo, detendría al modelo de ser tan eficiente con la reducción de dimensionalidad, dándole más organización a las representaciones en el espacio latente, pero a coste de tener una distribución menos eficiente. Esto genera un “trade-off” entre la importancia que se busque dar a la capacidad de generación del modelo, contra la capacidad de reconstrucción que tiene.

4. Metodología

4.1 Exploración y preprocesamiento de datos

Tras haber descargado el set de imágenes desde Kaggle, se realizó una exploración de estas imágenes y posteriormente se definieron 3 tamaños de imagen con los que se trabajaría en este proyecto: 100x100, 128x128 y 256x256 píxeles. Esta separación se realizó con el motivo de comprar resultados de imágenes de menor tamaño contra otras de mayor tamaño, así como también los tiempos de entrenamiento y cantidad de parámetros en el modelo.

El redimensionamiento de estas imágenes se almacenó en formato de NumPy (.npy) para poder acceder a ellos fácilmente desde los notebooks de entrenamiento de modelos. Finalmente, antes de entrenar el modelo, se realiza una normalización dividiendo entre 255 todas las imágenes, para que las matrices que entren a la red neuronal estén todas dentro del rango de cero a uno.

4.2 Desarrollo inicial de Variational Autoencoders

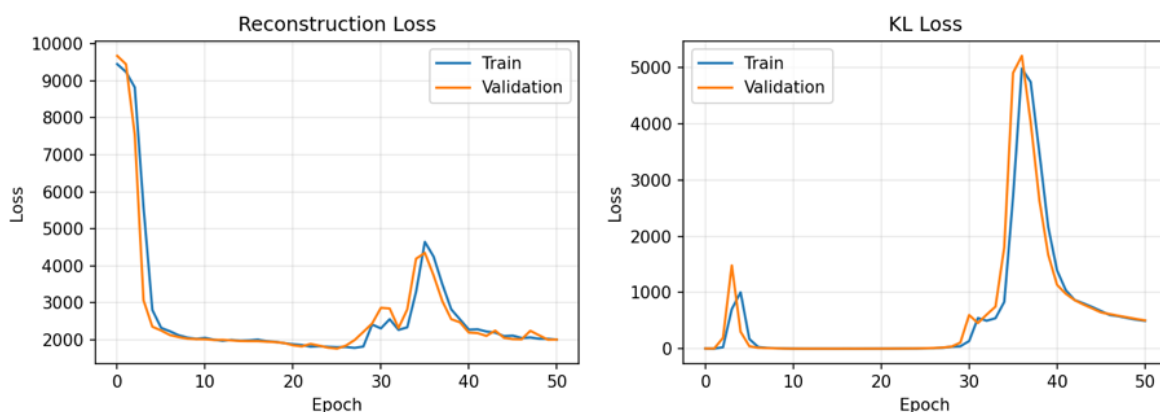
Una vez teniendo los datos de entrenamiento listos para usarse comenzó la fase de desarrollar las primeras arquitecturas de modelos. Se probaron gran cantidad de

arquitecturas diferentes para lograr mejorar los resultados, desde aumentar la dimensionalidad de las imágenes, hasta aumentar la cantidad de capas convolucionales antes de llegar al espacio latente del modelo.

Tras realizar muchas pruebas nos dimos cuenta de que para obtener los mejores resultados en nuestro modelo teníamos que implementar principalmente 4 componentes en nuestro modelo:

1. Capas convolucionales 2D con kernels tamaño 3x3 y un factor de reducción (strides) de 2, haciendo que los kernels salten 2 píxeles en cada movimiento.
2. Aplicación de Batch Normalization en cada capa convolucional del encoder y decoder, pues esto ayudó a mejorar la estabilización del entrenamiento y reducir el overfitting.
3. Capas de activación ReLU, pues tras experimentar con otras capas de activación fue la que logró mejores imágenes.
4. Parámetro $\beta = 1$. Tras probar valores de β en el rango desde 0.1 hasta 1.5, nos dimos cuenta que la forma en que el modelo aprendía mejor la distribución normal y producía mejores imágenes era utilizando $\beta = 1$.

Tras haber definido estos estándares para el entrenamiento de nuestros modelos, el siguiente punto importante en esta fase de experimentación fue el seguimiento de las funciones de pérdida durante el entrenamiento de modelos. Se utilizó una separación de los datos del 80% en entrenamiento y 20% en pruebas, a su vez, el set de entrenamiento se dividió un 20% para el set de validación utilizado en el entrenamiento. En este punto fue donde encontramos las mayores dificultades de este proyecto, pues parecía ser que el learning rate que estabilizaba a la pérdida de reconstrucción no era el mejor para la pérdida de KL-Divergence y viceversa.



Pérdida de reconstrucción y KL-Divergence en el primer modelo entrenado (Elaboración propia)

Como se puede ver en la imagen superior en donde se muestra la pérdida en la fase de entrenamiento en el primer modelo desarrollado, existe una gran inestabilidad, puesto que al iniciar el entrenamiento la pérdida de reconstrucción cae rápidamente y luego vuelve a subir. En cambio, el KL-Divergence inicia muy pequeño, sin embargo, posteriormente tiene picos altísimos, mostrando inestabilidad en el entrenamiento.

4.2 Arquitectura del modelo final

La arquitectura del modelo final es un VAE convolucional diseñado para imágenes de resolución 128x128 píxeles con 3 canales de color (RGB), pues finalmente fue la resolución que mayor resultado nos dio en relación calidad de imagen y tiempo de entrenamiento (por cantidad de parámetros en el modelo). El modelo cuenta con un encoder que parametriza la distribución latente un espacio latente de 28 dimensiones actuando como cuello de botella de información y un decodificador que reconstruye imágenes a partir de representaciones latentes.

4.2.1 Encoder

El encoder del modelo cuenta con 6 capas convolucionales secuenciales configuradas de la siguiente manera:

- Capa 1 Conv2D: 32 filtros de 3x3, stride=2. (128,128,3 → 64,64,32)
- Capa 2 Conv2D: 64 filtros de 3x3, stride=2. (64,64,32 → 32,32,64)
- Capa 3 Conv2D: 128 filtros de 3x3, stride=2. (32,32,64 → 16,16,128)
- Capa 4 Conv2D: 256 filtros de 3x3, stride=2. (16,16,128 → 8,8,256)
- Capa 5 Conv2D: 512 filtros de 3x3, stride=2. (8,8,256 → 4,4,512)
- Capa 6 Conv2D: 1028 filtros de 3x3, stride=2. (4,4,512 → 2,2,1028)

Cada capa convolucional va seguida de Batch Normalization para estabilizar el entrenamiento normalizando las activaciones y una función de activación ReLU pues como se mencionó anteriormente, fue la función de activación que mejores resultados dio en nuestros modelos. Seguido de las seis capas convolucionales se hace un aplanamiento que produce un vector de 4112 características, luego se reduce a una capa densa de 256 neuronas, dos capas densas paralelas de 28 neuronas cada una para parametrizar al media y varianza de la distribución latente. Finalmente se usa la técnica de reparametrización para obtener el vector latente $z = \mu + \sigma\epsilon$.

4.2.2 Decoder

Como se mencionó anteriormente en este reporte, el decoder realiza el proceso inverso, transformando los vectores del espacio latente nuevamente a imágenes de 128x128x3. Primeramente se toma una capa densa para proyectar del vector latente de 28 dimensiones a uno de 4112 características pasando por Batch Normalization y una capa de activación sigmoide, posteriormente comienza el proceso de reconstrucción, dónde al igual que en el encoder, cada capa convolucional cuenta con Batch Normalization y capa de activación ReLU:

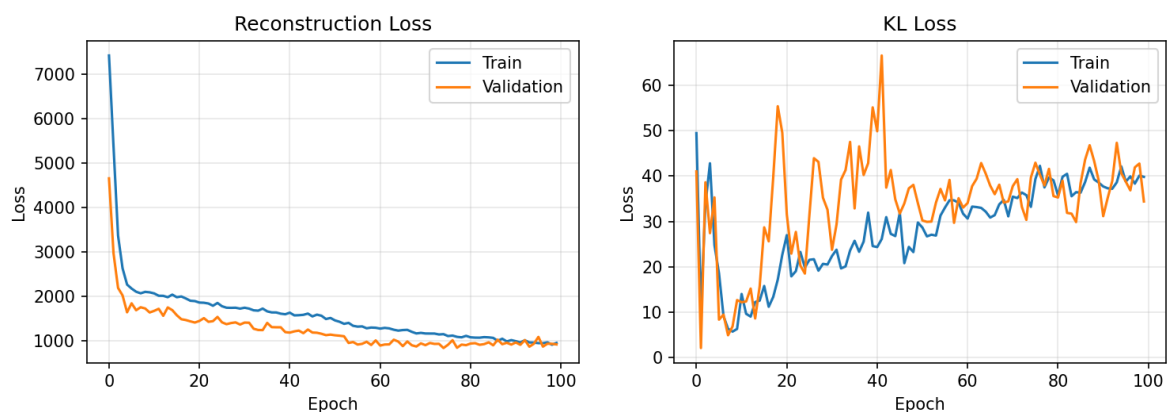
- Capa 1 Conv2DTranspose: 1028 filtros, stride=2. (2,2,1028 → 4,4,512)
- Capa 2 Conv2DTranspose: 512 filtros, stride=2. (4,4,512 → 8,8,256)
- Capa 3 Conv2DTranspose: 256 filtros, stride=2. (8,8,256 → 16,16,128)
- Capa 4 Conv2DTranspose: 128 filtros, stride=2. (16,16,128 → 32,32,64)
- Capa 5 Conv2DTranspose: 64 filtros, stride=2. (32,32,64 → 64,64,32)
- Capa 6 Conv2DTranspose: 32 filtros, stride=2. (64,64,32 → 128,128,3)

Al final, el modelo resultante, con la arquitectura descrita anteriormente tuvo una cantidad final de: 23,343,491.

5. Resultados

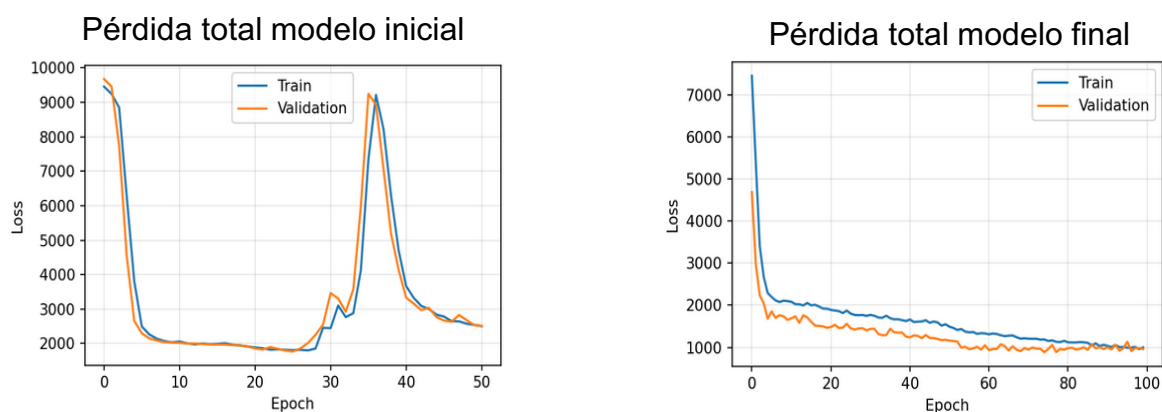
5.1 Funciones de pérdida

Tras la implementación de la arquitectura propuesta en la metodología se logró considerablemente reducir la pérdida que se tenía en los primeros intentos de modelo. Recordando la pérdida que se obtenía en el primero modelo entrenado, se tenía una pérdida de reconstrucción de aproximadamente 2,000 luego de 50 épocas de entrenamiento, mientras que en el modelo final se logró reducir y estabilizar considerablemente en alrededor de 1000, lo que significa que el modelo reducción la pérdida de reconstrucción un 50% aproximadamente. Esto puede verse en la siguiente gráfica:



Pérdida de reconstrucción y KL-Divergence en el modelo final (Elaboración propia)

Por otro lado, la pérdida de KL-Divergence presentó algo de inestabilidad, pues como se ve en la gráfica anterior, se muestran algunos picos, pues como mencionamos antes en el reporte, parecía ser que el learning rate que estabilizaba una pérdida no era compatible con la otra. No obstante, los valores de KL-Divergence se redujeron enormemente a pesar de la inestabilidad, pues en el modelo final se estabilizó alrededor de 40, mientras que en el modelo original se encontraba alrededor de 500, indicando una reducción del 92%, mejorando considerablemente el modelo.



Pérdida Total (Reconstrucción + β KL-Divergence) del modelo inicial vs final (Elaboración propia)

En cuanto a la pérdida total del modelo inicial podemos observar la alta inestabilidad, terminando en valores 2,500 aproximadamente, mientras que, en el último modelo, tras estabilizar el entrenamiento, no se observan altos picos, sino que se ve un descenso suave, terminando en valores de 1,000 aproximadamente, representando una reducción de la pérdida total de un 60%.

5.2 Reconstrucción de imágenes del set de pruebas

En la imagen inferior se muestra la reconstrucción que realizó el modelo final de imágenes del set de prueba, es decir, imágenes que el modelo nunca observó para el entrenamiento. Se observa que las motos 1, 2 y 4 tienen una reconstrucción buena,

pues se aprecian detalles de las motos originales sin un exceso de desenfoque. Por otro lado, las motos 3 y 5 no tuvieron la mejor reconstrucción, pues no se aprecian tantos detalles y la reconstrucción se muestra borrosa. Esto puede deberse a que la imagen reconstruida tiene el fondo con una tonalidad un poco más oscura.

Original: Superior | Reconstrucción: Inferior

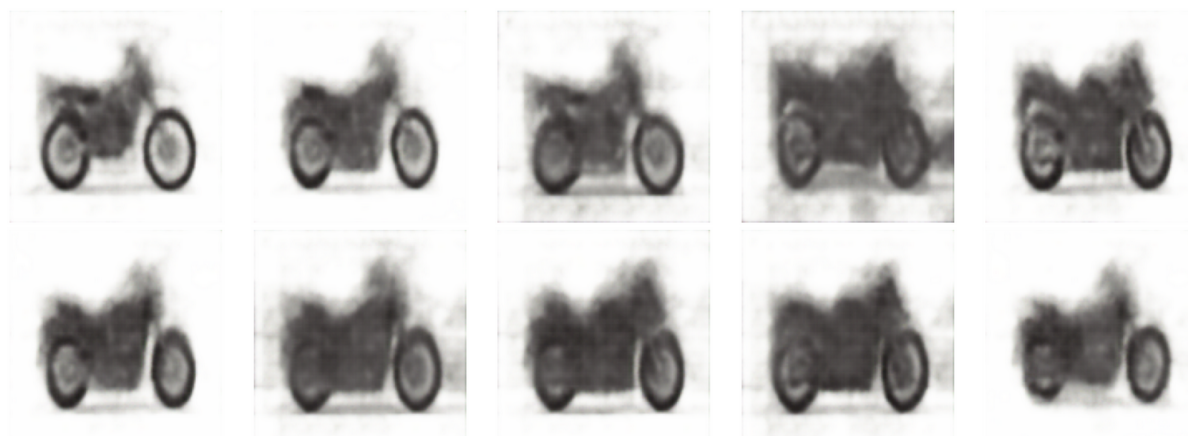


Reconstrucción de imágenes con el modelo final (Elaboración propia)

5.2 Generación de nuevas imágenes

Posteriormente, se realizó la generación de imágenes. Este proceso es posible, dado que al minimizar la pérdida de KL-Divergence hace que la distribución latente se acerque a una normal. Con ello es posible muestrear una distribución normal de tamaño ($\#n$ imágenes a generar, dimensión latente) y simplemente utilizar el decoder ya entrenado para predecir el muestreo realizado y terminar con una imagen en la salida.

Con este proceso generamos 10 imágenes que pueden observarse en la gráfica inferior. Al muestrear números aleatorios, en teoría deberíamos de observar motocicletas diferentes entre sí, puesto que el modelo debió de aprender diferentes rasgos de las motocicletas utilizadas para el entrenamiento. Poniendo en práctica esto, logramos observar que en efecto tenemos motos con diferentes rasgos, algunas con el asiento más alto o bajo, e incluso con cuerpos más robustos y delgados. Sin embargo, nuevamente se observa que las imágenes tienen ausencia de color y un poco de desenfoque.



Generación de nuevas imágenes (Elaboración propia)

5.3 Ausencia de color y desenfoque

Tras ver los resultados de reconstrucción y generación de imágenes, observamos que nuestras motocicletas presentan un leve desenfoque, difuminando rasgos y características de cada moto, y también ausencia de color, pues todas parecen estar en blanco y negro.

En cuanto a la ausencia de color en las imágenes podemos pensar que al entrenar el modelo con la función de pérdida de reconstrucción de MSE, la operación a realizar es un promedio, por lo que, al promediar gran cantidad de colores observados en motos distintas, se obtendrían tonos grisáceos, haciendo que el modelo solo aprenda a generar este color.

No obstante, estas mismas limitaciones son mencionadas por Jianing Liu, 2024, dónde concluye que los VAE producen imágenes borrosas y desenfocadas al momento de promediar con el MSE en la función de pérdida de reconstrucción, la cual puede ser problemática en tareas que requieran mucho detalle. Además, menciona que los VAE, a pesar de estar diseñados para generar diversas imágenes suelen tener problemas con salidas distintas comparado a otros modelos como las GANs. Finalmente menciona que los VAE enfrentan un problema de trade-off en cuanto a la reconstrucción y regularización del espacio latente, lo que tiende a minimizar más una función que la otra, lo que lleva al underfitting o una peor generalización. Este mismo comportamiento fue lo que enfrentamos en este proyecto, pues al estabilizar una función de pérdida, la otra sufría desestabilización.

6. Conclusión

El objetivo del proyecto se cumplió, como se evidencia en el reporte. La principal meta era diseñar una arquitectura que fuera capaz de generar imágenes usando como

marco inicial los Autoencoders Variacionales tradicionales, y sobre eso intentar mejorar la capacidad del modelo con variaciones de este marco, entre ellas el β -VAE.

Sin embargo, aún existen áreas de oportunidad para el proyecto. Por ejemplo, a pesar de que durante la metodología se implementaron técnicas de mejora como data augmentation, o mejorar la resolución de las imágenes para que el modelo obtuviera mejor información. También se llegaron a encontrar más modelos aparte de la implementación de beta, como Conditional VAE, VQ VAE, generative and adversarial VAE para construir modelos híbridos. Algunos hipotéticos que consideramos mientras que no se llegaron a investigar es la generación de una mejor representación del espacio latente como para demostrar la distorsión que se tiene al alejarse de los puntos de entrenamiento, o el recompensar en el modelo mejor la generación de color.

Gracias a este proyecto, pudimos explorar la generación de imágenes, utilizando auto-encoders variacionales

7. Referencias:

- Liu, J. (2025). Research on the application of variational autoencoder in image generation. ITM Web of Conferences, 70, 02001. <https://doi.org/10.1051/itmconf/20257002001>
- Newson, A. (n.d.). Autoencoders and generative models [Course materials]. MVA - DELIRES, Telecom Paris. Retrieved from https://perso.telecom-paristech.fr/newson/doc/courses/mva/mva_generative_models.pdf
- Michelucci, U. (2022, January 12). An introduction to autoencoders. arXiv preprint arXiv:2201.03898. <https://arxiv.org/pdf/2201.03898>
- Higgins, I., Matthey, L., Pal, A., Burgess, C., Glorot, X., Botvinick, M., Mohamed, S., & Lerchner, A. (2017). β -VAE: Learning basic visual concepts with a constrained variational framework. In Proceedings of the International Conference on Learning Representations. <https://www.cs.toronto.edu/~bonner/courses/2022s/csc2547/papers/generative/disentangled-representations/beta-vae,-higgins,-iclr2017.pdf>

8. Recursos:

Dataset obtenido de:

- <https://www.kaggle.com/datasets/imbikramsaha/caltech-101?resource=download>